

DiamondWorld: Validating Baseball Counterfactuals Against Market Forecast Changes

Luke Blommestein¹ and Jaden Majumdar²

¹Western University lblommest@uwo.ca

²University of Toronto jaden.majumdar@mail.utoronto.ca

MIT Sloan Sports Analytics Conference, Research Paper Track

Abstract

Introduction. Game-specific counterfactuals (what a lineup or starter change does to *this* game) are what teams actually decide on, but they are hard to validate: the counterfactual outcome is never observed, so a simulator can report any number and no realized result contradicts it. **Methods.** We build DiamondWorld, a plate-appearance-level generative model, an empirical rules engine, and a full game simulator, and we propose a way to check its counterfactual forecast changes without realized outcomes. Within a series the two teams and home field are fixed, so the game-to-game change in an independent pregame market’s implied win probability is driven by the starter and lineup. We test whether the simulator’s within-series forecast changes agree with the market’s, using no game outcomes and no look-ahead, and we calibrate the simulator’s magnitudes to the market on an earlier period and evaluate the frozen calibration on a held-out one. Every simulated input, including the bullpen, is selected from information available before first pitch. **Results.** Across 2,343 within-series game changes (646 series, 30 teams, 2024), the simulator’s and the market’s forecast changes correlate $r = 0.48$ (95% CI 0.43 to 0.53, cluster bootstrap by series; sign accuracy 0.67), on both pitching and hitting changes, and the calibration factor holds out-of-sample (frozen slope 0.98). The same machinery values individual starting pitchers: the simulator’s win probability added per start agrees with the market’s pricing of 175 starters at $r = 0.64$, and at $r = 0.51$ to 0.54 when the two are measured on disjoint starts. Its per-game run distribution beats a summed model on proper scores (log score 2.84 vs 2.94) and matches a league negative binomial while conditioning on the matchup. Its player layer, averaged with a Marcel-style projection, is statistically level with Steamer (0.668 vs 0.671). Point win probability is calibrated but not sharp, and the simulator is not intended to beat the market on generic prediction. **Conclusion.** We externally validate the direction and calibrated scale of game-specific baseball counterfactuals against an independent forecaster, a check that has, to our knowledge, not been reported before.

1 Introduction

Marginal projection systems such as Steamer, ZiPS, THE BAT, and Marcel [1] describe player skill accurately, but they do not by themselves provide game-specific counterfactual distributions. A front office decides on those: *what does acquiring this pitcher do to our playoff chances, does this lineup change the shape of tonight’s game, how confident should we be in this win probability.* A game is a correlated sequence of plate appearances (PAs), so its mean is a sum of rates but its *shape* is not, and a summed projection cannot answer these questions.

A generative game simulator can, by construction. The difficulty is not building one, which has been done for decades [2, 3]. It is trusting it. A simulator will report that swapping in an ace is worth a given number of win-probability points, but that number is internally generated and never observed: the counterfactual game is played once, with the actual roster. This paper’s contribution is a way to check such counterfactuals externally. We ask two questions. (Q1) *Can a game-specific simulator produce counterfactual forecast changes that agree with an independent pregame information aggrega-*

tor, without using realized game outcomes? (Q2) *Does the simulator provide a better conditional run distribution than independent or mean-only baselines?* A market’s movement is a forecast revision, not observed causal truth, so we treat market agreement as a *validation target*, not ground truth, and we are careful about that distinction throughout.

2 Related work

Player projection and simulation. Marcel [1] and its relatives regress recent performance toward a mean; Markov-chain models of baseball [2] and deep counterfactual simulators [3] produce run distributions and strategy estimates. A sufficiently rich Markov model can itself generate correlated, game-specific distributions; our contribution is not that simulation is new, but the external validation of its counterfactual sensitivities. **Game-state value.** Continuous game-state value models, notably expected possession value in basketball [4], estimate the value of a state from a stochastic process; we produce state-conditional estimates but validate discrete interventions against a market. **Markets as forecasters.** Pregame prices from consensus sportsbook moneylines and from pre-

diction markets (Kalshi, Polymarket), de-vigged to implied probabilities [5], are strong information aggregators; we use them as an external forecast to validate against, not as causal ground truth. **Evaluation.** We use proper scoring rules and probability-integral-transform calibration for count data [6, 7], classical win-probability baselines (Log5, Pythagorean) [8], and, where we discuss interventions, the potential-outcomes framing [9]. The model is fit by stochastic variational inference [10, 11, 12].

3 The DiamondWorld model

DiamondWorld is a pipeline of three components (Fig. 1). The **plate-appearance model** is a generative model fit by stochastic variational inference [10, 11]: for each PA it draws a nine-way categorical outcome (K, BB, HBP, 1B, 2B, 3B, HR, out, reached-on-error) from a softmax over a linear-plus-embedding score in batter and pitcher identity, a per-batter skill latent, park, and game state (inning, base-out state, score, times through the order, cumulative pitch count). Player identity enters through recency-weighted rate features and the skill latent; correcting a minibatch scaling of the latent’s divergence revived a posterior that had collapsed to its prior. The locked specification adds *contact-quality features*, a per-batter expected hit and home-run rate. These are constructed once, from training-season batted balls only: we bin batted balls by exit velocity and launch angle, take the league hit and home-run frequency in each bin, and average over each batter’s own batted balls to a static per-batter feature (Fig. 2). They are *not* the current PA’s launch measurements, so they are available before the PA; Table 1 lists every feature and its availability, and removing the contact-quality features reverts the model to its ablation (0.624 $\rightarrow$ 0.609 cross-player correlation), a small change rather than a collapse, confirming they are an additive, leak-free lever. A code audit then found and fixed three data-path defects (an outcome-label mismatch, an attention mask that let the first position see later ones, and unknown players sharing an index with a real one); with per-stat shrinkage of the rate features and a season-to-season random-walk prior on the skill latent, this raises cross-player correlation to 0.644 (paired gain +0.019 [+0.007, +0.031]).

One locked model. Every game-level result below uses this one specification, trained once (seed 42) and simulated at $R=500$ replicas per game. An independent retraining from scratch on a second machine reproduces its player differentiation (0.643 vs 0.644). We also trained fifteen variants on the corrected pipeline (other seeds, an optional sequence encoder over the game’s earlier plate appearances, optional joint training with a pitch-level model); none we scored paired against the locked model differs from it on player differentiation (differences from -0.005 to $+0.007$, none

Table 1. Feature availability audit. All model inputs are available before the plate appearance; no post-contact quantity from the current PA is used.

Feature	Pre-PA?	Construction
Batter / pitcher rate features	✓	prior-season rates
Skill latent (per player)	✓	learned, prior seasons
Contact quality (per batter)	✓	prior-season batted balls, static
Park, game state, TTO, pitch count	✓	pregame / in-sim state
Current-PA exit velo / launch angle	—	<i>not used</i>
Realized game outcome	—	<i>not used</i>

significant), so we did not select among them, and we report them only as a robustness check (Table 3).

The **empirical rules engine** maps an outcome sequence to runs and base-state transitions; ported from a validated Markov baseline [2], it reproduces test-season run totals to within half a percent. The **game simulator** plays full nine-inning-plus-extras games with lineup cycling, a bullpen, walk-offs, and the ghost-runner rule. Relievers follow a fitted *hook model*, a logistic hazard for pulling the starter in batters faced, times through the order, and runs allowed (held-out AUC 0.889), so the *timing* of the hook is endogenous: it uses only pregame-fitted parameters and in-simulation state. *Who* relieves is also chosen pre-game: reading the relievers off the completed game would leak the manager’s in-game choices, which are endogenous to how the game went, so each team’s bullpen is instead the pitchers who appeared alongside that game’s starter in strictly earlier games, ordered by how often they did. The starter and the batting order are announced before first pitch and are used as announced. An earlier version of these results used the realized bullpen; every game-level number reported here is leak-free. In-game state is generated recursively as the game is played. The model is trained through 2023 (rate features through 2024 for the later-season checks) and evaluated on 2024 unless stated. A held-out player-differentiation ablation of architectures is reported in Appendix A; the short version is that on that metric, features mattered far more than architecture.

4 What the simulator answers

Because the object is generative, four families of questions fall out directly (Table 2): single-input counterfactuals (re-simulate the same game with one input changed), lineup construction (reorder the same nine hitters), correlated tail-risk (the full game-total distribution, with a within-game dependence a summed model lacks), and win probability with a decomposition

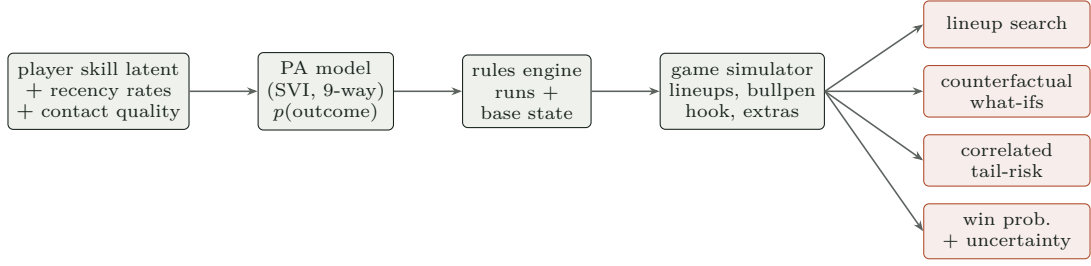


Figure 1. The DiamondWorld pipeline. A generative plate-appearance model provides outcome probabilities; a validated rules engine converts outcomes to runs and base states; a full game simulator adds lineups, an endogenous bullpen, and extras. All exogenous inputs are available before first pitch; in-game state (inning, base-out, score, times through the order, pitch count) is generated recursively by the simulator.

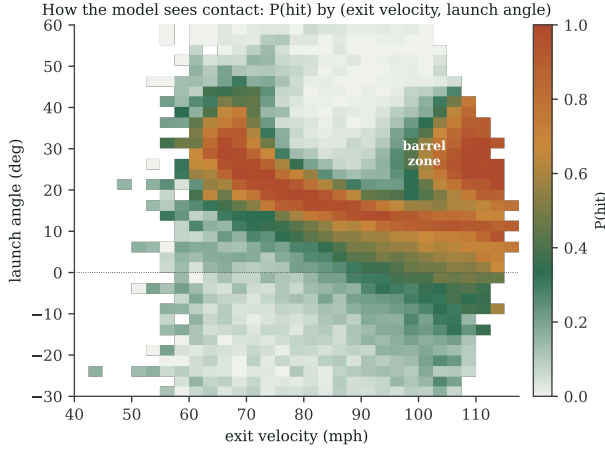


Figure 2. The contact-quality feature. League probability of a hit as a function of exit velocity and launch angle, from training-season batted balls. Each batter’s static feature is the average over *their own* batted balls; it is not the current PA’s launch data (Table 1).

of game randomness from roster uncertainty (comparable in size over 2024, 4.6% vs 4.9% WP). A rich Markov model can produce some of these; what is distinctive here is that the counterfactuals are player-specific, generative, and, below, externally validated.

5 Validating counterfactual forecast changes

The estimate “change this starter, win probability moves by X ” is internally generated. We check it against an independent pregame market’s implied win probability, from consensus sportsbook closing money-lines de-vigged to probabilities [5], for the same teams under different pitchers. Within a series the two teams and home field are fixed, so the game-to-game move in the market’s implied win probability reflects changes in the starter, lineup, rest, and other time-varying pregame information (Fig. 3); if the simulator’s within-series moves track the market’s, its counterfactual sensitivities have external support that uses *no realized*

Table 2. What each approach provides. A rich Markov model can generate correlated distributions; the distinctive claims here are player- and game-specific generative counterfactuals and their external validation.

Capability	Proj. sum	Markov (rich)	DiamondWorld
Player rate lines	✓		✓
Correlated run distribution		✓	✓
Uncertainty decomposition		partial	✓
Player-specific counterfactual		partial	✓
<i>externally validated</i>			✓

outcomes and no look-ahead. We stress that this validates the *direction and scale of forecast changes*, not causal effects: the market is an external forecaster, its move can reflect other same-day changes (rest, travel, weather, umpire, late lineup or bullpen news), and it may share public inputs with the simulator, which would make the agreement partly non-independent. We therefore report it as forecast-change agreement and quantify its uncertainty carefully.

Statistics (Table 3). On 2,343 within-series game deviations (646 series, 30 teams), the simulator’s and market’s forecast changes correlate $r=0.48$, with a 95% confidence interval of $[0.43, 0.53]$ from a cluster bootstrap that resamples whole series (Spearman $\rho=0.49$; the logit scale gives 0.47). Sign accuracy is 0.67 $[0.65, 0.69]$, well above chance. The result is significant far beyond a permutation null (95th percentile 0.03). The simulator explains close to a quarter of the market’s game-specific revision ($R^2=0.23$): a real but partial share, which is the honest reading. The result is not specific to the locked model. Every one of the fifteen variants is significant alone, and pooling all sixteen gives 0.50 $[0.44, 0.56]$, statistically indistinguishable from the locked model (paired difference $-0.02 [-0.05, +0.00]$). Replicas matter more than model choice: the same kind of model simulated at $R=100$ gives 0.31 to 0.38, because replica noise in each game’s forecast attenuates the correlation.

Table 3. Within-series validation, 2,343 game deviations in 646 series, leak-free staff, cluster-bootstrap 95% CIs by series. Direction agrees with the market and the calibration holds out-of-sample; the effect is real but partial ($R^2 = 0.23$). R is replicas per game.

Simulator	R	r [95% CI]	Sign
Locked model, probability scale	500	0.48 [.43,.53]	0.67
Locked model, logit scale	500	0.47 [.41,.53]	0.67
<i>Robustness:</i> all 16 variants pooled	800	0.50 [.44,.56]	0.69
same recipe, 2 independent runs	200	0.35 [.30,.41]	0.61
one variant at low replicas	100	0.31 [.26,.37]	0.61

Locked model: OLS slope 0.37 [.34,.41], intercept ≈ 0 , MAE 0.027. Pitching / hitting channel t : $-17.6 / +7.3$ (both priced). *Calibration fit on half the series, frozen:* held-out slope 0.37, corr 0.46, sign 0.67, MAE 14% below predict-zero; frozen-calibrated slope 0.98 (want 1), intercept ≈ 0 .

Calibration separated from evaluation, and the magnitude holds out-of-sample. The magnitude-calibration slope (0.37 raw, 0.45 after correcting for the simulator’s replica-sampling noise, reliability 0.83 at $R=500$) is fit on one random half of the series and evaluated, frozen, on the held-out half. The held-out half is not only directionally right (sign accuracy 0.67); its magnitude matches. Its own OLS slope is 0.37 (essentially the 0.38 fit on the other half) with intercept ≈ 0 and correlation 0.46, and the decisive check is that regressing the held-out market change on the *frozen-calibrated* simulator change gives slope 0.98 (target 1) and intercept ≈ 0 : the calibration transfers rather than being refit on what it is evaluated on. The held-out mean absolute error is 14% below a predict-zero baseline. The raw simulator overstates market-scale moves by about a factor of two, so a raw “+9.8-point” swing on a representative game becomes about +4.4 points in market-calibrated units.

Both pitching and hitting are priced. Decomposing each within-series move into a pitching channel (starters’ allowed-run rate) and a hitting channel (lineup rate value), both carry independent signal the market responds to (pitching $t=-17.6$, hitting $t=+7.3$), so the validation is not pitcher-only.

A later season, and a stale-input stress test. Freezing the rate features and repeating the within-series test on the 2025 sportsbook consensus, a season the features predate, gives $r=0.33$ (channel-level, using the model’s inputs rather than a full re-simulation; see Limitations). As a deliberate stress test we also run 2026-to-date against a *prediction market* (Kalshi), a different market mechanism, with rates now 2.5 years stale (only about six of nine batters and half the starters known): agreement attenuates to $r=0.12$, as expected. We do not claim this as clean generalization; it shows the signal is real, appears across a sportsbook and a prediction market, and degrades with stale in-

Table 4. The three evaluations differ in what is actually exercised; only the 2024 result is a full end-to-end simulation.

Evaluation	Season	What is tested
End-to-end simulator	2024	full simulation counterfactuals ($r=0.32$)
Channel replay	2025	frozen-model input sensitivities ($r=0.33$)
Stale-input stress	2026	input sensitivity with 2.5-yr-stale rates ($r=0.12$)

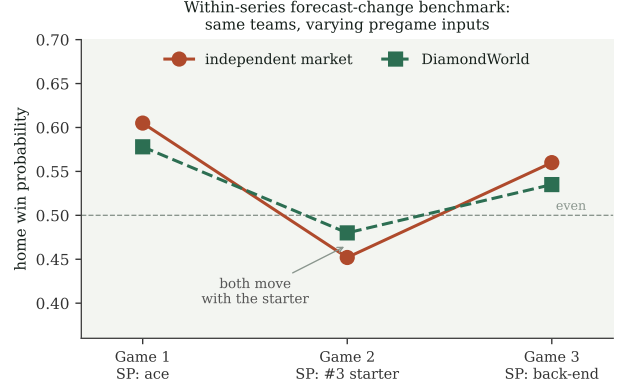


Figure 3. The within-series design (illustrative). With the two teams and home field held constant, the game-to-game move in win probability reflects the change in starter, lineup, rest, and other pregame inputs. The validation measures whether the simulator’s move (green) tracks the independent market’s (orange).

puts. A second prediction market (Polymarket) carries the same game markets but its price histories did not pin cleanly to a single game’s first pitch for a bulk pregame harvest, so we report it as assessed rather than as a third estimate (see Reproducibility). These three evaluations are not the same test and we keep them separate (Table 4): only 2024 is a full end-to-end simulation; 2025 replays the frozen model’s input sensitivities without re-simulating the season, and 2026 is a stale-input stress test.

6 From forecast agreement toward a causal reading

The within-series design is a first difference that removes both team fixed effects and home field, so it is closer to an identified intervention than a raw correlation. Under a potential-outcomes reading [9], for an ordered series we compare the simulator’s *do*(swap game t ’s roster for game $t+1$ ’s) estimate against the market’s differenced forecast, and we stress the identifying assumption three ways. (i) *Rest control.* Adding the within-series difference in days of rest leaves the simulator coefficient essentially unchanged (0.373 $\rightarrow$ 0.372, $t=26$), so the effect is not a rest con-

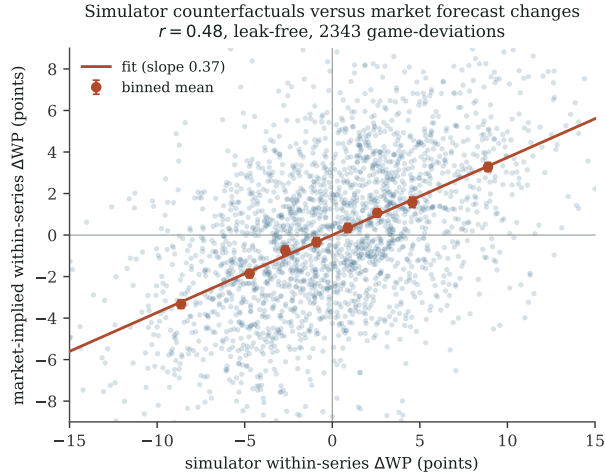


Figure 4. Simulator vs. market within-series win-probability deltas, 2,343 game-deviations (locked model, $R=500$, leak-free staff). Team strength and home field are differenced out. $r=0.48$; the slope gives the magnitude calibration.

found. (ii) *Negative-control placebo.* Pairing each simulator delta with a market delta from a random other series gives mean correlation 0.00 (99th percentile 0.05); the observed 0.48 is far outside that distribution. (iii) *Sensitivity to unobserved confounding.* An omitted differenced covariate would have to correlate at least 0.69 with *both* the simulator and market deltas to explain the association; rest, the strongest observed such covariate, manages only 0.07. So no single plausible confounder accounts for the effect. We are explicit that this is a design-based identification with stated assumptions, not a timestamp-clean starter- scratch event study: the announced-probable-pitcher data required for that is not publicly recoverable (the free schedule API overwrites the announced probable with the actual starter post-game, leaving only four apparent 2024 scratches, which are artifacts). A clean event study with proprietary intraday lines is the natural next step.

7 Run distribution and where the simulator does not lead

A validated counterfactual engine is only useful if its raw outputs are realistic, so we benchmark them and report where the simulator trails (Table 5).

Run-total distribution. Against an independent two-Poisson model with the identical per-game means (only the shape differs), the simulator matches reality’s spread (within-game total variance $2.07\times$ the Poisson value, real $2.11\times$), with strictly better proper scores (log score 2.84 vs 2.94, paired difference -0.102 [$-0.125, -0.078$]; CRPS 2.39 vs 2.45, paired difference -0.061 [$-0.076, -0.046$] [6, 7]; negative favors the sim-

ulator) and better tail probabilities (Fig. 5). It covers near nominal rates (achieved 80% coverage 0.84 [0.82, 0.85], nominal 0.80) where Poisson badly under-covers (0.67), and this holds *conditional on the pregame expected total* (0.82 to 0.88 across quartiles vs Poisson 0.65 to 0.71), so the dispersion is game-conditional, not a single unconditional variance matched on average. We do not claim to beat every baseline: a marginal league negative-binomial matches the simulator on aggregate proper scores (log-score difference -0.003 [$-0.019, +0.013$], CRPS difference $+0.016$ [$-0.010, +0.041$], both straddling zero), but it assigns every matchup the identical distribution, which the simulator’s game-specific conditional distributions do not.

Win probability. Point win probability is honestly not the simulator’s edge: it is calibrated (ECE 0.026) and beats the home-field base rate (log loss 0.683 vs 0.692), but it is less sharp than a one-line team-strength baseline (AUC 0.58 vs Log5 0.62). A per-PA process washes team talent toward a coin flip, and Diamond-World is not intended to beat a market on generic prediction; its value is the conditional distribution and the validated counterfactuals.

Player realism. On 376 batters with at least 150 PA in 2024, the corrected model’s cross-player rate correlation is 0.648, level with a Marcel projection augmented with the same contact-quality feature (0.651) and below Steamer’s real preseason release (0.671). The two are complementary: a parameter-free average of their standardized predictions reaches 0.668, statistically level with Steamer (paired difference -0.002 [$-0.021, +0.016$]) and matching it on home runs (0.654 vs 0.651), while beating each of its parents ($+0.020$ [$+0.009, +0.032$] over DiamondWorld alone, $+0.017$ [$+0.005, +0.029$] over Marcel). A fitted blend does worse (0.660), because least squares targets squared error rather than correlation, so we report the unfitted average, which cannot overfit (Fig. 7). The simulator itself consumes the model’s rates, not the blend; the blend shows that the model’s player layer carries information a Marcel-style projection lacks.

8 Decision studies

The game-level validation says the simulator’s forecast *changes* are right in direction and, once calibrated, in scale. A front office acts on a different object: the value of a specific player. We run two decision studies, one on a lever the market prices and one on a lever it does not.

What is a starter worth per start? Rotation order, skipping a start, and above all trading for a starter turn on one number: how much win probability a pitcher adds when he takes the ball, relative to the alternatives on his own staff. The simulator answers it directly, and the within-series design gives it an ex-

Table 5. Benchmarks. The simulator beats the game-independent two-Poisson baseline on the run-total distribution and matches the marginal negative-binomial on aggregate proper scores while providing game-specific conditional distributions the marginal cannot; it trails a one-line team-strength baseline on point win probability and is competitive on player realism. Bold marks the best in each row (both marked when a difference is within noise). Bracketed values are 95% bootstrap CIs over games.

Metric	DiamondWorld	Independent / summed	Team-strength	Reference
Run-total log score (lower better)	2.84	2.94 (2-Poisson)		2.84 (marginal NB)
Run-total CRPS (lower better)	2.39	2.45 (2-Poisson)		2.38 (marginal NB)
Run-total 80% coverage	0.84 [.82,.85]	0.67 (2-Poisson)		0.80 (nominal)
conditional on expected total	0.82–0.88	0.65–0.71 (2-Poisson)		0.80 (nominal)
Win probability AUC	0.58		0.62 (Log5)	0.61 (market)
Player rate corr. (avg)	0.648		0.651 (Marcel+CQ)	0.671 (Steamer)
averaged with Marcel+CQ	0.668			0.671 (Steamer)

Paired score differences (DiamondWorld minus baseline, negative favors DiamondWorld): vs 2-Poisson, log score -0.102 $[-0.125, -0.078]$ and CRPS -0.061 $[-0.076, -0.046]$; vs marginal NB, log score -0.003 $[-0.019, +0.013]$ and CRPS $+0.016$ $[-0.010, +0.041]$ (both straddle zero). Averaged player layer minus Steamer: -0.002 $[-0.021, +0.016]$.

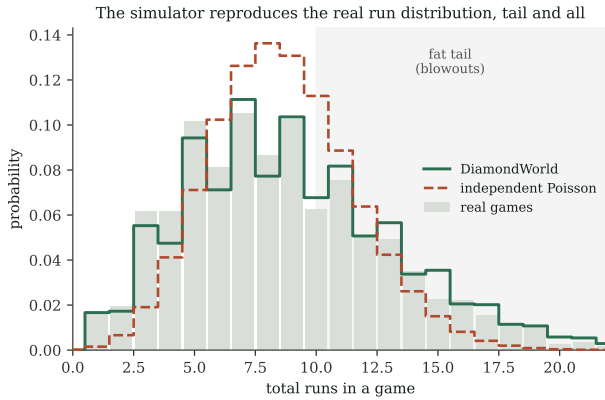


Figure 5. Game-total run distribution. The simulator (green) matches real games (bars) including the fat blowout tail; an independent-Poisson model with the same mean (dashed) is too peaked and too thin-tailed.

ternal check. In each series we express every game’s forecast as a deviation from the series mean, which removes both teams and home field; we credit the home starter with the deviation and the away starter with its negative, so each credit is from the pitcher’s own team’s point of view; a pitcher’s value is his mean credit, computed separately from the simulator and from the market. Across the 175 starters with at least ten 2024 starts (3,976 starts), the simulator’s values agree with the market’s at $r=0.64$ $[0.53, 0.74]$ (Fig. 8). Because both are averages over the same games, the stricter test measures them on disjoint starts: the simulator’s value from a random half of a pitcher’s starts predicts the market’s value from the other half at $r=0.54$ $[0.42, 0.64]$ and, with the halves swapped, 0.51 $[0.38, 0.63]$. The market agrees with itself across halves only at 0.79 , so the simulator recovers most of what is recoverable. A calibration factor fit on half the pitchers and frozen on the other half has slope 0.86 (target 1) and intercept

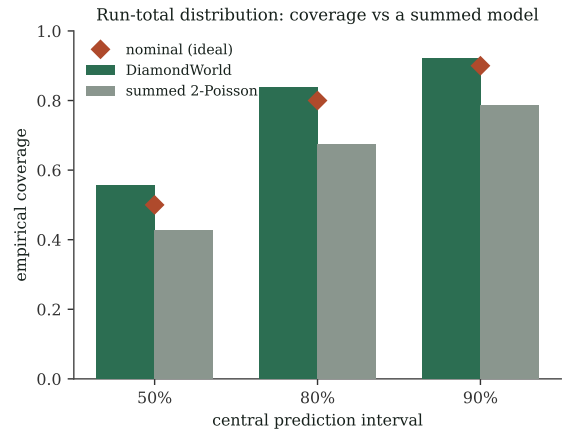


Figure 6. Interval coverage over 2,376 games (bootstrap CIs in Table 5). The simulator is slightly conservative and stays so conditional on the pregame expected total; the summed model badly under-covers.

-0.14 points, and cuts the error of a predict-zero baseline by 27%. In calibrated units the top of the 2024 list is what a front office would expect (Burnes $+3.1$ points per start against the market’s $+3.6$, Glasnow $+4.3$ vs $+5.5$, Cole $+2.6$ vs $+2.8$), and over a 32-start season $+3$ points per start is about one win above the rest of the pitcher’s own rotation. The misses are informative rather than random. Five of the eight largest overvaluations are veterans aged 31 or older whose prior-season rates had not yet caught up with their decline (Paxton, Maeda, Pérez, Walker, Verlander; the model has no aging curve). Four of the eight largest undervaluations had little or no major-league history before 2024: rookies Skenes and Jones, and the Japanese signings Yamamoto and Imanaga, whom the model sees only through a league-average prior; two more are breakouts (Skubal and Ragans). Aging and pre-MLB evaluation are precisely the inputs a team supplies from its own

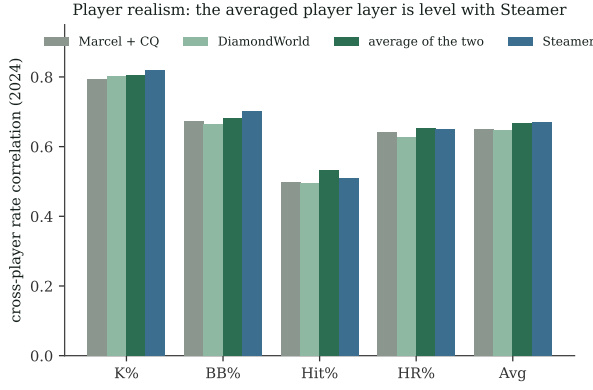


Figure 7. Cross-player rate correlation on 376 batters, 2024. DiamondWorld alone is level with a Marcel projection given the same contact-quality feature; the unfitted average of the two is statistically level with Steamer (0.668 vs 0.671).

scouting, and the calibrated simulator is the layer that turns them into wins.

Lineup construction. Our second study takes a lever the market does not price, with the winner’s curse controlled. For each 2024 game we generate candidate batting orders of the home nine, *select* the order maximizing expected runs on one set of replica seeds, then *evaluate* the selected, actual, and a random order on an *independent* set of seeds. The selection/evaluation seed split is essential: the best of many candidates is inflated on its own selection seeds, so we never score a choice on the seeds used to pick it. Honestly, a counterfactual lineup’s realized outcome is unobservable, so the reported gain is the simulator’s own unbiased estimate of the lever. We are careful about what is validated by what: the market evidence in §5 validates the general counterfactual machinery, whereas the lineup number below is an application produced by that machinery and is not itself market-validated. The two claims are kept separate.

Across 50 held-out 2024 games with 12 candidate orders each, the naive gain measured on the selection seeds is +0.47 runs per game, which annualizes to an implausible +76 runs per season. Scored on independent evaluation seeds, that gain collapses to −0.03 runs per game (95% cluster CI [−0.12, +0.06]), with only half of games improved and the selected order beating a random order by +0.003 versus the actual order’s +0.032. The corrected estimate is statistically indistinguishable from zero. We read this as two findings. First, the winner’s-curse correction is doing real work: without the selection/evaluation seed split we would have reported a large, entirely illusory improvement. Second, batting-order optimization is a near-null lever at this granularity, consistent with the sabermetric consensus that lineup order is worth at most a few runs across a full season. The methodological point of

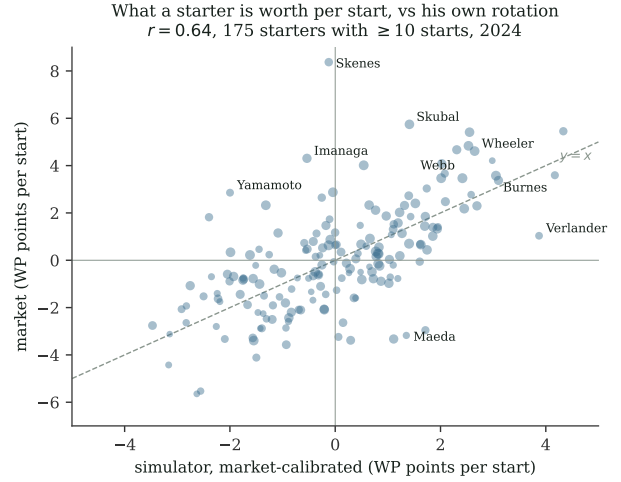


Figure 8. Win probability added per start relative to the pitcher’s own rotation, simulator (market-calibrated, x) vs market (y), for 175 starters with at least ten 2024 starts. $r = 0.64$; $r = 0.51$ to 0.54 when the two are measured on disjoint starts.

the study is not that the simulator wins a decision, but that the same discipline used for the market validation prevents it from manufacturing one. Together the two studies mark the boundary of use: the simulator is trustworthy on levers the market can check, such as who pitches, and on levers it cannot check it should be trusted only as far as a winner’s-curse-controlled estimate allows. (The lineup study was run on an earlier version of the simulator.)

9 Limitations

We are explicit about the boundaries. (1) The market is an external forecaster, not causal ground truth; its move can reflect other same-day changes and it may share public inputs with the simulator, so the $r=0.48$ is forecast-change agreement, not an identified causal effect. (2) $R^2=0.23$: the agreement is real but partial. (3) The 2025 and 2026 checks use the model’s rate features rather than a full re-simulation of those seasons, and the 2026 slice is a stale-input stress test on an incomplete season. (4) Raw counterfactual magnitudes overstate by about $2\times$ and must be market-calibrated. (5) Point win probability is less sharp than a one-line baseline. (6) The simulator’s mean total is 0.45 runs per game high (9.08 vs 8.63); the proper-score comparisons mean-match before scoring, so they test shape, and the offset largely cancels in within-series differences. (7) The replica count matters: at $R=100$ the same kind of model scores 0.31 to 0.38, so the headline requires simulating enough replicas per game. (8) The starter valuations inherit the model’s blind spots: no aging curve, and only a league-average prior for pitchers new to the major leagues.

10 Discussion

The contribution is a way to *externally validate the direction and calibrated scale of game-specific baseball counterfactuals* when the counterfactual outcome cannot be observed, together with a conditional run distribution that beats an independent baseline on proper scores and keeps its coverage conditional on expected runs. To our knowledge, we did not find prior work that validates game-specific baseball counterfactual sensitivities against independent pregame market forecast changes. For a front office the implication is concrete: the simulator’s market-calibrated value of a starting pitcher per start agrees with how the market prices him, on starts it did not see, so acquisition and rotation what-ifs rest on a validated engine, while levers the market does not price, such as batting order, come with a winner’s-curse-controlled estimate and no more. Its player layer is Steamer-grade when averaged with a simple projection, and the known gaps (aging, pitchers new to the major leagues) are exactly the inputs a team’s own scouting supplies. All code and data-join pipelines are released (Appendix B).

A Architecture ablation

We compared the generative model against a per-PA MLP, a GRU, a causal transformer, and a joint-embedding predictive architecture (JEPA) [13, 14] with variance-covariance regularization [15] and a frozen probe, on identical inputs and target (Fig. 9, Fig. 10). Held-out per-PA log-likelihood is a poor yardstick: a PA is near the empirical entropy of the league-average outcome distribution (about 1.495 nats, well below the nine-category maximum of $\log 9 \approx 2.197$ nats), so all models score near that floor. The sharper metric is cross-player differentiation. A frozen JEPA probe attains the best likelihood and near-zero differentiation; fine-tuning its encoder recovers it ($0.06 \rightarrow 0.54$) and from-scratch supervised training does marginally better, so the collapse is the frozen objective and the self-supervised pretraining adds nothing here. Once differentiation is measured correctly, the transformer, GRU, and MLP cluster near 0.577 and features (contact quality) move the number more than architecture. Metric definitions, seeds, and per-run values are in the repository.

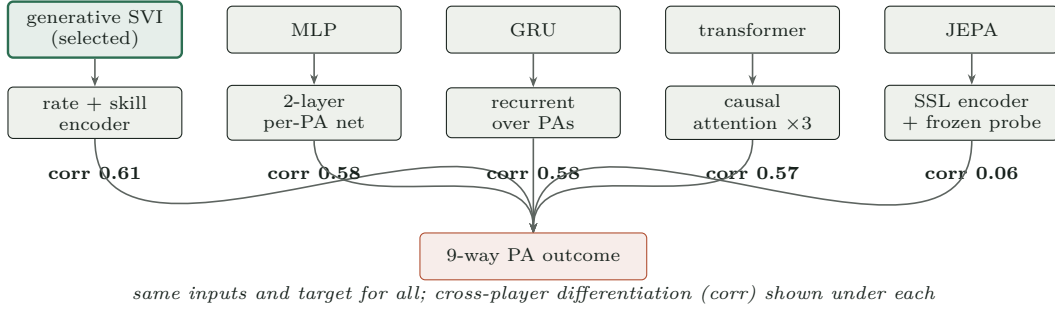


Figure 9. Architectures evaluated on identical inputs and target. The generative model with contact-quality features leads; discriminative nets cluster near 0.58; a frozen self-supervised probe collapses to 0.06 despite the best likelihood.

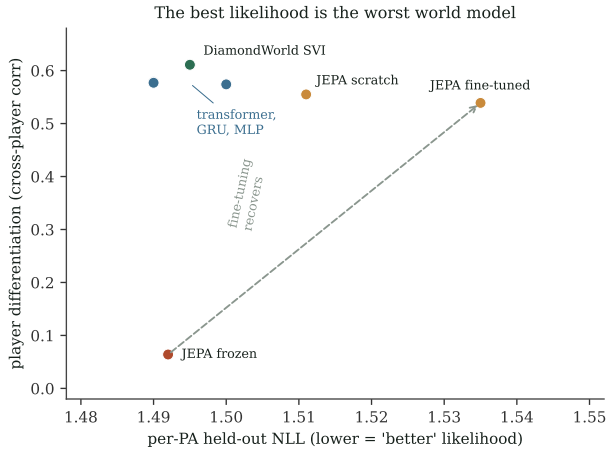


Figure 10. Held-out likelihood versus player differentiation. A frozen self-supervised probe minimizes NLL and cannot tell players apart; fine-tuning the same encoder recovers differentiation.

B Reproducibility

The repository contains the model, simulator, market-join pipelines, and every figure, with fixed seeds, an environment lockfile, a data dictionary with source URLs and as-of dates, train/validation/test manifests, rules-engine tests, and a one-command reproduction of the main figures and tables. Market inputs that cannot be redistributed are provided as derived pregame probabilities with the exact de-vigging and join logic and source timestamps, and the restriction is stated rather than papered over. Key scripts: `scenario_sim.py`, `run_pregame_sim.py`, `simulator_benchmarks.py`, `validation_stats.py`, `distributional_stats.py`, `counterfactual_validation.py`, `whatif_channels.py`, `season_market_validation.py`, `lineup_backtest.py`, `make_figures.py`.

References

- [1] T. Tango. The Marcel the Monkey Forecasting System. `tan-gotiger.net`, 2004.
- [2] B. Bukiet, E. R. Harold, and J. L. Palacios. A Markov chain approach to baseball. *Operations Research*, 45(1):14–23, 1997.
- [3] H. Nakahara, K. Takeda, and K. Fujii. Estimating the effect of team hitting strategies using counterfactual virtual simulation in baseball. *arXiv:2206.01871*, 2022.
- [4] D. Cervone, A. D’Amour, L. Bornn, and K. Goldsberry. A multiresolution stochastic process model for predicting basketball possession outcomes. *J. Amer. Statist. Assoc.*, 111(514):585–599, 2016.
- [5] E. Štrumbelj. On determining probability forecasts from betting odds. *Int. J. Forecasting*, 30(4):934–943, 2014.
- [6] T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *J. Amer. Statist. Assoc.*, 102(477):359–378, 2007.
- [7] C. Czado, T. Gneiting, and L. Held. Predictive model assessment for count data. *Biometrics*, 65(4):1254–1261, 2009.
- [8] B. James. *The Bill James Baseball Abstract*. Ballantine Books, 1984.
- [9] G. W. Imbens and D. B. Rubin. *Causal Inference for Statistics, Social, and Biomedical Sciences*. Cambridge Univ. Press, 2015.
- [10] M. D. Hoffman, D. M. Blei, C. Wang, and J. Paisley. Stochastic variational inference. *J. Mach. Learn. Res.*, 14:1303–1347, 2013.
- [11] D. Phan, N. Pradhan, and M. Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv:1912.11554*, 2019.
- [12] J. Bradbury, R. Frostig, P. Hawkins, et al. JAX: Composable transformations of Python+NumPy programs, 2018.
- [13] Y. LeCun. A path towards autonomous machine intelligence. OpenReview, 2022.
- [14] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, pages 15619–15629, 2023.
- [15] A. Bardes, J. Ponce, and Y. LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.